

- Description

This module manages inventory in a smart warehouse, ensuring that:

1. Products **must have valid stock levels** (no negative inventory).
2. When an order is placed, **the requested quantity must not exceed available stock**.
3. If stock runs low (below a threshold), **an automatic restocking order is triggered**.
4. The system maintains **a transaction history of stock movements**.

This ensures efficient inventory management with automated stock replenishment.

- Specification

module SmartWarehouseInventory

var stock, quantity: integer

var restockThreshold, restockAmount: integer

var transactionHistory: sequence of string

process processOrder(quantity: integer) flag: boolean

functional scenario form:

quantity <= stock && flag = true and stock = stock – quantity and transactionHistory = transactionHistory $\oplus$ ("Order processed") and if stock < restockThreshold then stock = stock + restockAmount, Valid order, stock updated, check for restocking

||

quantity > stock && flag = false, Invalid order, insufficient stock, values unchanged

||

quantity <= 0 && flag = false, Invalid order, quantity must be positive, values unchanged

end process

end module

- Java Code

```
package code;
import java.util.ArrayList;
import java.util.List;
class Product {
    private String name;
    private int stock;
    private final int restockThreshold = 5;
    private final int restockAmount = 20;
    private List<String> transactionHistory;
    public Product(String name, int initialStock) {
        this.name = name;
        this.stock = initialStock;
        this.transactionHistory = new ArrayList<>();
    }
}
```

```

    }
    public boolean processOrder(int quantity) {
        if (quantity > 0 && quantity <= stock) {
            stock -= quantity;
            transactionHistory.add(String.format("Order processed: -%d
%s, Remaining stock: %d", quantity, name, stock));
            System.out.printf("Order successful! %d %s shipped. Remaini
ng stock: %d\n", quantity, name, stock);
            if (stock < restockThreshold) {
                restock();
            }
            return true;
        } else {
            System.out.println("Order failed! Invalid quantity or insuf
ficient stock.");
            return false;
        }
    }
}
private void restock() {
    stock += restockAmount;
    transactionHistory.add(String.format("Auto-
restocked: +%d %s, New stock: %d", restockAmount, name, stock));
    System.out.printf("Auto-
restocking triggered! Added %d %s. New stock: %d\n", restockAmount, nam
e, stock);
}
public void printTransactionHistory() {
    System.out.println("Transaction History for " + name + ":");
    for (String log : transactionHistory) {
        System.out.println(log);
    }
}
public int getStock() {
    return stock;
}
}
public class c07_SmartWarehouse {
    public static void main(String[] args) {

        // testcase-VT:
        //     Product Laptop1 = new Product("Laptop", 10);
        //     Laptop1.processOrder(13); // quantity > 0 && quantity > stock
        //     Laptop1.printTransactionHistory();
        //     Product Laptop2 = new Product("Laptop", 10);

```

```

//      Laptop2.processOrder(19); // quantity > 0 && quantity > stock
//      Laptop2.printTransactionHistory();
//      Product Laptop3 = new Product("Laptop", 10);
//      Laptop3.processOrder(2); // quantity > 0 && quantity < stock
&& NO restock
//      Laptop3.printTransactionHistory();
//      Product Laptop4 = new Product("Laptop", 10);
//      Laptop4.processOrder(7); // quantity > 0 && quantity < stock
&& restock
//      Laptop4.printTransactionHistory();
//      Product Laptop5 = new Product("Laptop", 10);
//      Laptop5.processOrder(-4); // quantity < 0
//      Laptop5.printTransactionHistory();

      // testcase-FT:
//      Product Laptop1 = new Product("Laptop", 10);
//      Laptop1.processOrder(251);
//      Laptop1.printTransactionHistory();
//      Product Laptop2 = new Product("Laptop", 10);
//      Laptop2.processOrder(-72898649);
//      Laptop2.printTransactionHistory();
//      Product Laptop3 = new Product("Laptop", 10);
//      Laptop3.processOrder(6);
//      Laptop3.printTransactionHistory();
//      Product Laptop4 = new Product("Laptop", 10);
//      Laptop4.processOrder(1645);
//      Laptop4.printTransactionHistory();
//      Product Laptop5 = new Product("Laptop", 10);
//      Laptop5.processOrder(109);
//      Laptop5.printTransactionHistory();

      // testcase-Z3:
//      Product Laptop1 = new Product("Laptop", 10);
//      Laptop1.processOrder(16); // quantity > 0 && quantity > stock
//      Laptop1.printTransactionHistory();
//      Product Laptop2 = new Product("Laptop", 10);
//      Laptop2.processOrder(9); // quantity > 0 && quantity > stock
//      Laptop2.printTransactionHistory();
//      Product Laptop3 = new Product("Laptop", 10);
//      Laptop3.processOrder(11); // quantity > 0 && quantity < stock
&& NO restock
//      Laptop3.printTransactionHistory();
//      Product Laptop4 = new Product("Laptop", 10);

```

```
//      laptop4.processOrder(9); // quantity > 0 && quantity < stock
&& restock
//      laptop4.printTransactionHistory();
//      Product laptop5 = new Product("Laptop", 10);
//      laptop5.processOrder(11); // quantity < 0
//      laptop5.printTransactionHistory();

// testcase-UVT:
Product laptop1 = new Product("Laptop", 10);
laptop1.processOrder(13); // quantity > 0 && quantity > stock
laptop1.printTransactionHistory();
Product laptop2 = new Product("Laptop", 10);
laptop2.processOrder(19); // quantity > 0 && quantity > stock
laptop2.printTransactionHistory();
Product laptop3 = new Product("Laptop", 10);
laptop3.processOrder(2); // quantity > 0 && quantity < stock &&
NO restock
laptop3.printTransactionHistory();
Product laptop4 = new Product("Laptop", 10);
laptop4.processOrder(7); // quantity > 0 && quantity < stock &&
restock
laptop4.printTransactionHistory();
Product laptop5 = new Product("Laptop", 10);
laptop5.processOrder(-4); // quantity < 0
laptop5.printTransactionHistory();

}
}
```

● Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	808	816
✓ src	1.0 %	8	808	816
✓ code	1.0 %	8	808	816
✓ c07_SmartWarehouse.java	100.0 %	8	0	8
> c07_SmartWarehouse		0	0	0
> Product	100.0 %	8	0	8

3. Vibration testing instruction coverage: 97.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.5 %	226	15,284	15,510
✓ src	1.5 %	226	15,284	15,510
✓ code	1.5 %	226	15,284	15,510
✓ c07_SmartWarehouse.java	97.4 %	226	6	232
> c07_SmartWarehouse	95.3 %	61	3	64
> Product	98.2 %	165	3	168

4. Fuzz testing branch coverage: 87.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.9 %	7	809	816
▼ src	0.9 %	7	809	816
▼ code	0.9 %	7	809	816
▼ c07_SmartWarehouse.java	87.5 %	7	1	8
> Product	87.5 %	7	1	8
> c07_SmartWarehouse		0	0	0

5. Fuzz testing instruction coverage: 97.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.5 %	226	15,284	15,510
▼ src	1.5 %	226	15,284	15,510
▼ code	1.5 %	226	15,284	15,510
▼ c07_SmartWarehouse.java	97.4 %	226	6	232
> c07_SmartWarehouse	95.3 %	61	3	64
> Product	98.2 %	165	3	168

6. Z3 branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	834	840
▼ src	0.7 %	6	834	840
▼ code	0.7 %	6	834	840
▼ c07_SmartWarehouse.java	75.0 %	6	2	8
> Product	75.0 %	6	2	8
> c07_SmartWarehouse		0	0	0

7. Z3 instruction coverage: 97.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.0 %	226	21,860	22,086
▼ src	1.0 %	226	21,860	22,086
▼ code	1.0 %	226	21,860	22,086
▼ c07_SmartWarehouse.java	97.4 %	226	6	232
> c07_SmartWarehouse	95.3 %	61	3	64
> Product	98.2 %	165	3	168

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	808	816
▼ src	1.0 %	8	808	816
▼ code	1.0 %	8	808	816
▼ c07_SmartWarehouse.java	100.0 %	8	0	8
> c07_SmartWarehouse		0	0	0
> Product	100.0 %	8	0	8

9. Unimproved V-method instruction coverage: 97.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.5 %	226	15,284	15,510
▼ src	1.5 %	226	15,284	15,510
▼ code	1.5 %	226	15,284	15,510
▼ c07_SmartWarehouse.java	97.4 %	226	6	232
> c07_SmartWarehouse	95.3 %	61	3	64
> Product	98.2 %	165	3	168